蚂蚁·密码学实验室二面小抄

老板熟 MPC：SHIELD 的「需求 → 选择」故事 + masking + ML-KEM

自用，不共享屏幕

投递简历：v2-crypto / Changqi_Sun_AntGroup_PQC2027.pdf（不改，口头更正）　AI Coding 笔试：暂不做

- 页序：一面回顾 → 开场 → 纠错总表 → **1 SHIELD** → **2 Falcon/FLOCK** → **3 masking** → **4 HERO** → **5 threshold ML-KEM** → **6 概念讲解 E1–E6** → 术语 + 数字速查
- 红字【待确认】= 推断或没核实，面试前自己扫一遍

一面回顾（王宇辰，11-04，近 2 小时）

被问到的

- HERO 讲了很多（按时间顺序讲的，没问题）
- SHIELD：**为什么 malicious? 为什么 SPDZ? 为什么 WRK17?**
  当时答「导师推荐 / SPDZ 好做 malicious」——不够 convincing
- **每个人都问性能**：最终能做到多少 latency、几方
- 面试官提到 **ISW** ——我没接住（见 E1）
- 简历第 1 条 masking + 省内存：还没投，要能说清状态

对方透露的

- 老板熟 MPC，会追问设计取舍
- 组里想做 **threshold ML-KEM**，但自己也不确定能不能做 $\Rightarrow$ 我的定位：**可行性探索**（Part 5）
- 他提到 threshold ML-DSA 有好几篇新工作——见「相关工作」页

本轮要纠正的记忆

- 最多测到 **5 方**（不是 6/8；6 是 Celi/Borin 的上限）
- 「$<$ 500 ms」是**单次迭代**；87@10 ms RTT 是 530 ms

开场 30 秒（中文说，术语英文）

- VT CS 四年级 PhD，导师 Thang Hoang，方向 applied crypto，2028-06 毕业。数学本科。
- 主线：**lattice 签名的 threshold 化**，而且坚持输出就是标准签名：
  - SHIELD（S&P 2026 共同一作 + 报告人）：threshold ML-DSA，MPC 里跑**不改动**的 FIPS 204
  - FLOCK（一作，USENIX Sec 2027 在投）：Falcon-compatible threshold signature
- 实现侧：masked ML-DSA on Cortex-M4 ——发现寄存器级防护在 bus / port 上漏，并把峰值 RAM 降 37–42%
- 另有 ORAM 工作 HERO（一作，NDSS 2027 在审）
- 一句收尾：**「我做的事情都是：在不改标准算法的前提下，把它放进更苛刻的环境里（多方 / 侧信道 / 小内存）」**——正好接 threshold ML-KEM

先读这页：我之前说错 / 记错 / 没意识到的（逐条改口）

我说过 / 记得的	正确说法（怎么说出口）
「masking 本质上就是 MPC in the head」	MPC-in-the-head 是用 MPC 造 ZK proof（IKOS07）。masking 应说「单设备上的 MPC：ISW private circuits，probing model 下每个中间值拆成 $d+1$ 份」——见讲解页 E1/E2
「WLK30 / WRK 那篇」	WRK17 = Wang–Ranellucci–Katz, "Global-Scale Secure Multiparty Computation", CCS 2017（authenticated garbling 的多方版）
「最多做到 6 或 8 方」	SHIELD 测了 2–5 方。6 方是 Celi et al. 的上限
「online $<$ 500 ms」	单次成功迭代 198–536 ms（87@10 ms RTT 超 500）；端到端还要乘期望重试 $\approx$4 次
「Efficient threshold ML-DSA $<$ 100 ms」	Celi et al.（USENIX Sec'26）本机 0.5–12 ms、LAN $<$ 20 ms、跨洲 WAN 0.6–0.75 s；「sub-100 ms」出自未评审的 Kao arXiv
「double shares，每方存两份」	short replicated secret sharing；每方存 2 份只是 $(N, T) = (3, 2)$ 的特例，一般是 $\binom{N-1}{N-T}$ 份
「用 SPDZ 是因为比较容易实现 malicious」	SPDZ 用 IT-MAC 在 dishonest majority 下给 active security with abort ——关键词是 dishonest majority，不是「容易」

先读这页（续）：没意识到的限制 + 术语

我说过 / 记得的	正确说法（怎么说出口）
「用它是因为导师推荐」	按 R1–R4「需求 → 选择」讲（SHIELD 第 2 页）
「SHIELD 抗 $n-1$ 恶意」（没意识到的限制）	online 才是；offline 的 triples / daBits 由可信 dealer 生成
简历「890K→355K vs SCALE-MAMBA」	口头说「相对我们最初的朴素编码」；和 SCALE-MAMBA 比的是加法器 375→294
masking：「MakeHint / Keccak 都是 ISW 掩码」	MakeHint 明文算（检查全过并合并后）；Keccak 用 TI（threshold implementation）不是 ISW；论文不声称 SNI/PINI
「TI 的 threshold」	和 threshold signature 的 threshold 无关：TI 指「每个输出份只依赖部分输入份」（non-completeness）
「strict ML-KEM」	是 threshold ML-KEM；组里也不确定能不能做 ⇒ 我的定位是「可行性探索」
「Efficient Threshold ML-DSA = ePrint 2025/1166」	1166 已撤回、拆分；现在是 ePrint 2026/013（另一半 Threshold Raccoon 是 2026/419）

Part 1 ·SHIELD

threshold ML-DSA（S&P 2026）——老板重点

SHIELD 是什么（一页讲清）



- 场景：**FPKI**（联邦 PKI）——多个 CA 共同签发证书，任何单个 CA 都拿不到可用私钥
- 威胁模型：malicious、static、QPT 敌手，最多腐化 $n-1$ 个 CA；security-with-abort；quantum-UC 证明 model.tex:47, security_analysis.tex:52
- 门限：n-**of**-n（$t=n$）——「所有 CA 都参与，最大化 trust 和 accountability」proposed_method.tex:2
- 引擎：**SPDZ**（算术：NTT、$A\mathbf{y}$、$c\mathbf{s}_1$）+ **WRK17**（布尔：SHAKE/Keccak-f、ExpandMask、Decompose/HighBits、SampleInBall、CheckNorm、MakeHint）+ **daBits** 跨域桥接
- 实现：SPDZ 自己从零写；WRK17 用 EMP-toolkit；布尔电路用 Verilog → Yosys → Bristol format
- 定位一句话：前人做到 *functional interchangeability*（标准 verifier 能验），我们是 algorithmic FIPS-204 compliance（算法本身没改）background.tex:17

为什么这么选：先讲需求，再推出选择（核心页）

需求 / 约束	推导	选择
R1 持钥方本身就是敌手（CA 各自独立、互不信任）	半诚实不够：恶意方在 $\mathbf{z}$ 的份额或 reject bit 上加 δ，就能让本该被拒的 $\mathbf{z}$ 被打开——而**拒绝采样正是 ML-DSA 藏 $\mathbf{s}_1$ 的机制**；还有 selective failure、nonce 偏置	**malicious security**（IT-MAC 认证每个中间值）
R2 n-of-n, 要容忍 $n-1$ 个坏 CA	这就是 dishonest majority ⇒ 排除 Shamir / honest-majority RSS（Quorus 要 $n = 2t+1$，3 方只容 1 个坏人）	**dishonest-majority MPC**
R3 输出必须和 FIPS 204 **逐比特一致**，不改算法	排除「每方本地拒绝采样」(Celi：换成 hyperball 分布)、「加噪声 e_w」(Quorus)、「hash 放 MPC 外 + 同态承诺」(Tang) ⇒ 只能把**整个 Sign 原样搬进通用 MPC**	**通用 MPC**，不是定制协议
R4 ML-DSA 是混合计算：$\mathbb{Z}_q$ 多项式算术 + 大量位运算	全 SPDZ：比较 / 位分解在算术域要 $O(\log q)$ 轮；全 GC：23-bit 模乘上千 AND 门 ⇒ 各做擅长的	**SPDZ + WRK17 + daBits**

口径：「最初是导师建议的组合，但我回头复盘，它其实是这四条约束推出来的唯一自然解；如果放宽 R2 或 R3，就会走向 Quorus 或 Celi 那样更快的路线。」

为什么 SPDZ、为什么 WRK17（被追问细节时）

SPDZ（Damgård et al. 2012）

- 份额 $[\![x]\!]_i = (x_i,\ m_i)$，$\sum m_i = \alpha x$，$\alpha$ 是全局 MAC key
- 线性运算本地免费；乘法用 Beaver triple，只需打开 d, e
- online 信息论安全，开销约半诚实版的 2 倍
- 任意 N，容忍 $N-1$ 恶意——正好满足 R2
- ML-DSA 的 $q = 8380417$ 是素数 $\Rightarrow$ 主体就是 $\mathbb{F}_q$ 线性代数，和 SPDZ 天然契合
- 联合拒绝（不是每方各自拒绝）$\Rightarrow$ 期望重试次数 = 单机 ML-DSA（4.25 / 5.1 / 3.85）

WRK17（Wang–Ranellucci–Katz, CCS'17, "Global-Scale MPC"）

- authenticated garbling：预处理生成带 IT-MAC 的 AND triple（TinyOT 风格）
- malicious、dishonest majority（$N-1$），**online 常数轮**（我们是 4 轮），与电路深度无关
  experiment.tex:64
- 用在：SHAKE/Keccak-f、ExpandMask、Decompose/HighBits、SampleInBall (Fisher–Yates)、CheckNorm、MakeHint、A2B 加法器
  design_rational.tex:15–64
- 电路优化：SampleInBall 890K $\rightarrow$ 355K gates **【待确认】**简历写成「vs SCALE-MAMBA」；论文里是相对**我们最初的朴素编码**。口头说「相对最初编码」。和 SCALE-MAMBA/EMP 比的是 32-bit 加法器 375 $\rightarrow$ 294 门（-27%）

daBits：同一个随机比特在算术域和布尔域各有一份认证 share（Rotaru–Wood, Marbled circuits）$\Rightarrow$ A2B / B2A 只需打开一次掩码值

性能：数字要说准

单次成功迭代的 online latency（ms），$n = 2 \to 5$
本地集群 + 模拟 RTT：Low≈1 ms / Med≈3 ms / High≈10 ms，10 Gbps

	Low	Med	High
ML-DSA-44	198–205	253–258	414–418
ML-DSA-65	246–252	294–298	455–461
ML-DSA-87	321–326	369–376	530–536

experiment.tex:263–276

每个 CA 的 online 开销：23 轮；7.7 / 10.9 / 14.7 MB（+ 少量 $\propto n$）；内存 160–280 MB
吞吐：16 核并行，44@Low ≈20 sig/s，44@High ≈6 sig/s

怎么说（主动讲清口径）

- 「单次迭代 online 0.2–0.33 s（LAN），10 ms RTT 下 ML-DSA-44/65 不到 0.46 s；**最多测到 5 方**」
- 「端到端要乘期望重试 ≈4 次；可以像 Celi 那样并行跑 K 份来换 latency」
- 「没测真实 WAN；23 轮 online 在 100 ms RTT 下光等待就 2.3 s/迭代——这是 SPDZ 路线的代价」
- **offline 很重**：WRK17 预处理是瓶颈；摊销后一次成功迭代按分钟计（44@Low 2.8–8.6 min） perf_full_table.tex:139–153

对比：Celi 本机 0.5–12 ms、LAN < 20 ms、跨洲 WAN 0.6–0.75 s；Quorus online 计算 6–343 ms（不含网络）

软肋 Q&A（老板最可能追问，主动承认 + 给改进）

追问	回答
offline 谁做?	SPDZ 的 triples / daBits / AGV 由**可信 dealer**（setup 那个 CA）生成，所以 dishonest majority **只覆盖 online**。改进：MASCOT / Overdrive(LowGear) 生成 triples，edaBits（Escudero et al. CRYPTO'20）；WRK17 预处理本来就是各方自己跑的
MAC soundness?	$\mathbb{F}_p$ 上单次 check 伪造概率 $\approx 1/p$。若 $p = q \approx 2^{23}$ 只有 2^{-23} **【待确认】**论文只写 $\mathbb{F}_p$；A2B 按 $\lvert q\rvert = 23$ bit 计，推测 $p = q$。回去查代码确认。修法：k 个独立 MAC key（2^{-23k}）或扩域 MAC
WAN / 轮数?	23 轮 online，RTT 敏感。Celi 每次尝试 3 轮；这是「不改算法」的代价
后量子?	quantum-UC 只覆盖 IT-MAC 部分；WRK17 / EMP 的 base OT 通常是 ECC，不是 PQ。修法：base OT 换 PQ OT，OT extension 本身是对称密码
abort 能识别谁吗?	不能。只有 security-with-abort，static corruption。identifiable abort 要额外机制（SPDZ-IA 类）
被拒迭代的 opening?	**【待确认】**自查：被拒迭代里打开过的值是否也进 batch MAC check? selective failure 要求「影响后续行为的值」在用之前被认证
并发安全（加分）	已处理 Kyster et al.（S&P'25 "Rushing at SPDZ"）：并发下 MAC key 可被提取 $\Rightarrow$ 对所有 partial openings 做一次 batch MAC check design_rational.tex:75–77

相关工作：threshold ML-DSA 地图

工作	出处	技术	安全	方数	性能
Celi et al.「Efficient Threshold ML-DSA」(= NIST MPTC "Mithril")	USENIX Sec'26, ePrint 2026/013	short RSS，每方本地 hyperball 拒绝；每次尝试 3 轮	malicious, dishonest maj.; game-based; abort 不可识别	< 6 (8 勉强)	本机 0.5–12 ms；LAN $<$ 20 ms；WAN 27 ms（亚洲内）/ 0.6–0.75 s（跨洲）
Quorus (Bienstock et al., JPMC)	USENIX Sec'26, 2025/1163	Shamir mod q + edaBits；加噪声 e_w 使公开被拒 w_1 安全	malicious, honest maj.; UC	任意(测到 63)	online 计算 6–343 ms，不含网络；16–29 轮/次
Trilithium	ESORICS'26, 2025/675	2PC + BeDOZa MAC + 诚实的关联随机数提供方	需 CRP 诚实	2	秒级；CRP 每次 55 MiB
Tang et al. （最接近）	TIFS'23, 2024/1067	t-of-n SPDZ + 分布式拒绝采样；hash 在 MPC 外 + 同态承诺	主动安全	测到 12	0.5 s (2 方)，7.3 s (12 方)
SHIELD（我们）	S&P'26	SPDZ + WRK17 + daBits，整个 Sign 原样进 MPC	malicious, dishonest maj.; quantum-UC	2–5	单次迭代 0.2–0.54 s

口径：「和 Tang 的区别：他们 hash 放 MPC 外、用同态承诺，输出不是精确 ML-DSA；我们 SHAKE 也在 MPC 里，bit-exact。和 Celi 比我们慢两个数量级，但他们改了拒绝分布、只到 6 方。」

不要拿 Kao (arXiv 2601.20917) 做对比——未评审，同作者的 TALUS 已被 Niot (ePrint 2026/1386) 攻破

Celi et al. 的「double shares」= short replicated secret sharing

怎么分

- 对每个大小 $N-T+1$ 的子集 I，采一个**短的** $\mathbf{s}_I \sim \chi_s$；$\mathbf{s} = \sum_I \mathbf{s}_I$
- 参与者 i 持有所有 $i \in I$ 的 $\mathbf{s}_I$
- 总份数 $\binom{N}{T-1}$；每方 $\binom{N-1}{N-T}$ 份
- **(3,2)：共 3 份，每方 2 份** ← 「每人存两份」；(6,4)：共 20 份，每方 10 份
- 至多 $T-1$ 方被腐化 ⇒ 至少一个 $\mathbf{s}_I$ 全在诚实方手里

为什么必须「短」：Shamir 份额在 $\mathbb{Z}_q$ 上均匀、Lagrange 系数大 ⇒ 部分签名范数大，没法本地拒绝。RSS 重构系数是 $0/1$ ⇒ 每方部分秘密本身就短

怎么签

- 每方本地跑 FS-with-aborts：$\mathbf{z}_i = c\mathbf{s}_i^{\text{part}} + \mathbf{r}_i$，加起来就是签名
- 各方都要通过 ⇒ 成功率 $\approx p^T$ ⇒ 换成 **hyperball 拒绝**（ℓ_2，像高斯一样紧），imbalanced 半径（hint 只作用在 $\mathbf{z}^{(2)}$）
- 两阶段：各方本地拒绝（安全必需）+ combine 时全局检查（尺寸兼容）
- 承诺用完整 MLWE 样本 $\mathbf{w}_i = A\mathbf{r}_i$，被拒也能安全公开
- 3 轮/次：commit $H(\mathbf{w}_i)$（防 rushing）→ 公开 $\mathbf{w}_i$ → 发 $\mathbf{z}_i$；$K = \lceil 1/p \rceil$ 份并行

和我们的本质差别：输出能过标准 verifier，但签名分布不是 FIPS 204 的；份数随 N 指数涨 ⇒ $N \leq 6$

如果现在重做 SHIELD（加分页）

- **offline 去掉可信 dealer**：triples 用 MASCOT/LowGear，daBits → edaBits（一次拿一整串比特，A2B 更省）
- **latency**：并行跑 K 个候选（Celi 的 $K = \lceil 1/p \rceil$ 思路），一次成功率 $\geq 1 - e^{-1}$；批量 MAC check 合并所有候选
- *t*-**of**-*n*：SPDZ 本身支持 *t*-out-of-*n* 变体（Tang 那条路线）；FPKI 之外的托管 / 钱包场景需要它
- **MAC soundness**：多 MAC key 或扩域，让统计安全到 40–64 bit
- **场景拆分**：如果能接受 honest majority（例如同一机构的 3 台 HSM），直接走 Quorus / Shamir 更快；如果 $N \leq 6$ 且不要求分布一致，走 Celi
- 一句话：**「SPDZ+WRK17 是在最强约束下的解；约束一放松，就该换更快的定制协议。」**

Part 2 ・Falcon

threshold-Falcon artifact 修复 + FLOCK

Falcon 两条（简历第 2 条 + SUMMARY 里的 FLOCK）

threshold-Falcon artifact 修复（merged PR）

- 复现 CCS'26 "Thresholdizing Standardized FALCON" 时，签名**连续**多次 l2bound 失败——按 Falcon 的拒绝率统计上不可能 ⇒ 定位到 sampler
- 根因：emp-tool `PRG::random_data` 字节/元素单位不匹配 ⇒ sampler 缓冲区 **7/8 读的是未初始化堆内存**，签名范数偏大 ≈15%
- 第二个 bug：`read()/write()` 返回值存进 `size_t`，-1 变 SIZE_MAX，I/O 流静默失步
- 修后：均方范数与期望差 0.4%，零拒绝，受影响的 run 快 $\sim 9\times$；作者合并

FLOCK（一作，USENIX Sec'27 在投）

- 难点：Falcon 的 ffSampling 是浮点、$2n$ 次顺序采样、每次以秘密为中心 ⇒ 前人 threshold Falcon 每签名上万轮
- (1) 换 sampler：Peikert offline/online + Zalcon + Antrag trapdoor ⇒ 纯整数、可并行，顺序深度 2
- (2)「以秘密为中心」的采样放到 MPC 外：打开 masked center $c + e$，各方明文采样，回到 MPC 只做线性运算
- 结果：9 轮 online；4 方 LAN 25 ms vs Garg et al. 24.2 s；签名过 Falcon verifier（$-4/12$ forgery bits, $+9/32$ bytes）
- 对外说 **Falcon-compatible**，不说 "standard Falcon"

Part 3 ・masking

Clean Registers, Leaky Buses（在写，S&P 2027）

masking 总览：同一个 ML-DSA，从「多台机器」换成「一块芯片」

high-level 一句话：「SHIELD 把 ML-DSA 拆给多台机器算；masking 是**在一块 Cortex-M4 上把每个中间值拆成几份**，probing model 下也是一种 MPC（ISW private circuits）。两边的难点一模一样：算术域和布尔域之间的转换。」

论文："Clean Registers, Leaky Buses: Hardening Masked ML-DSA under Memory Constraints"，基于 mldsa-native 的一阶掩码签名路径；目标 S&P 2027 Cycle 2（摘要 11-10 / 全文 11-17）

只合并（unmask）这几样：每次尝试的 w_1、$\tilde{c}$；每行范数检查 1 个判定位；$\mathbf{r}, \mathbf{z}$ 只在三项检查都通过后才合并；**被拒尝试的 $\mathbf{z}, \mathbf{r}$ 永不合并**（FS-with-aborts 只保证被接受的那组可模拟）

份额数（每个都有理由）

域	份数	理由
算术 mod q $(\mathbf{y}, \mathbf{w}, \mathbf{z}, \mathbf{r}, \hat{\mathbf{s}}_{1,2})$	2	一阶；$A\mathbf{y}$、$c\mathbf{s}_1$ 对秘密线性，A, c 公开 $\Rightarrow$ 逐份额算，不要随机数
布尔：Keccak + $\mathbf{y}$ 采样器	4	针对 Weakest Link（Hermelink et al.）打 $\mathbf{y}$；他们建议升到 2 阶，**4 份是我们自己的选择**
布尔：Decompose + 两个范数检查	3	Balasch et al.：n 份在 transition 泄漏下只有 $\lfloor (n-1)/2 \rfloor$ 阶 $\Rightarrow$ 3 是一阶的最小值；代价 +97.46% 指令
t_0	0	FIPS 204 §6.1：不必当秘密

A2B / B2A：两个域怎么转（你关心的那块）

B2A：ExpandMask $\rightarrow \mathbf{y}$（4 布尔份 $\rightarrow$ 2 算术份 mod q）

- 做法（BBEFG18 §5「减随机份额再合并」）：抽随机算术份 $s \rightarrow$ 在 4 个布尔份上用 SecAdd 算 $a_0 = \gamma_1 - z - s \bmod q$（再一次 csubq）$\rightarrow$ **只合并** a_0，(a_0, s) 就是算术份
- SecAnd：4 份非完备 TI，3 个 fresh 字
- 备选（代码里非默认）：SPOG19 按位 B2A$_q$，每系数 54 次乘法

A2B$_q$：Decompose / HighBits（2 算术份 $\rightarrow$ 3 布尔份）

- Barthe et al. 18：每个算术份各自布尔拆分，用 **Kogge–Stone SecAdd** 相加，再 $n-1$ 次 csubq（mod q 不是 2^k，所以要条件减）
- HighBits 在布尔份上算（ISW AND + Kogge–Stone；乘常数用移位加法链）；只合并 3 个布尔份得 w_1；$w_0 = w - 2\gamma_2 w_1$ 留在算术域

范数检查 MChkNorm$_3$：份 0 加 $B-1$ $\rightarrow$ A2B $\rightarrow$ 加公开常数取最高借位 $\rightarrow$ ISW AND 累积各行 $\rightarrow$ **只 XOR 合并最后 1 bit**

没掩码 / 没用的（别说错）

- MakeHint 明文算（检查全过并合并后）；没有 1-bit B2A
- Goubin A2B 没用：值模型成立，**transition 模型不成立**
- Keccak 的 χ：4 份 TI + changing of the guards（不是 ISW/DOM）；随机数 28.8 KB $\rightarrow$ 2.9 KB / 置换
- 不声称 SNI / PINI，只有一阶、值模型的逐 gadget 论证

代价：Kogge–Stone 每次加法 10 SecAnd + 9 refresh（ripple 要 31+31）；真板每次尝试 ≈27k 次 Keccak-f、3.7 MB 随机数，掩码 RNG 占 65.9% 时间

和 SHIELD 对照：SHIELD 用 daBits 一次打开掩码值做转换（MPC 里可以「打开」）；芯片上没人能安全「打开」，只能用 SecAdd 电路在份额上算 $\Rightarrow$ 贵

泄漏发现：寄存器干净，总线漏

机制：2 份 refresh 写成 `a0^=r`；`a1^=r`，两份从没先后进同一个寄存器，但**先后经过数据总线**；布尔份 $\mathrm{HD}(x_0, x_1) = \mathrm{HW}(x) \Rightarrow$ 直接漏明文的 HW

- 同一个二进制：寄存器模型 $\max|t| \le 5.83$（干净）；总线模型 chknorm / decompose $|t| = 199.00$
- **R1**：同一秘密的两份不能在同一个有状态位置（寄存器 / 总线 / 操作数端口）相邻
- **R2**：每个 barrier 必须清并声明**所有**可分配寄存器

-O2 vs -O3：barrier 只清 caller-saved r0–r3/r12，-O2 下编译器把相邻份放进 callee-saved r6/r8 $\Rightarrow$ decompose 24 点超阈，$\max|t|$ 14.10；改全寄存器 barrier（r0–r12）后消失

对策：bus-and-port barrier（栈字写 0 再读回 + 全 0 寄存器跑 `mla` 驱动端口）+ 3 份检查路径 + 全寄存器 barrier；+14.65% 指令

评估：Cortex-M4 仿真，register / bus / operand 三种模型的一、二阶 TVLA，全签名（最多 2.675 亿条指令 / trace），CPA + template，各带 positive control；TVLA 阈值随 trace 长度：$\sqrt{2\ln(2m/\alpha)}$，$\alpha = 0.05$

诚实说的限制

- 没上真机测泄漏；对两个公开 M4 硬件数据集检出 27/46、26/37
- 3 个 FAIL 原样记录：template GE 15.97 $<$ 16；种子 Keccak 1 点 6.14 $>$ 6.08（最多漏份 1）；-O2 rhoprime 序言/尾声
- 只做一阶；没做水平攻击（LAZY_Y 让同一 $\mathbf{y}$ 算 3 次）

省内存的逻辑（ML-DSA-44：50,612 → 31,828 B，-37.1%）

步骤	做法 / 为什么安全	Δ	峰值
d 默认掩码	—		71,900
n0 加固 +LAZY	LAZY_A：$\hat{A}$ 逐元素现算（ρ 公开，−15,264 B）；LAZY_Y：$\mathbf{y}$ 从 $\langle\rho''\rangle_4$ 重采（−6,072 B，但同一 $\mathbf{y}$ 多次暴露）		50,612
M1	公开的 t_0 打包存	−2,424	48,188
M6a	删掉掩码路径从不读的 NTT($\mathbf{y}$) 暂存区	−3,072	45,116
M6b	明文 $\mathbf{r}$ 写进已死的掩码 $\mathbf{w}$ 存储 【待确认】覆写曾存份额的字：字节粒度写 0 在一个设备上漏，未排除	−4,096	41,020
M7	明文 $\mathbf{z}$ 行放进 $\hat{A}$ 槽位（生命周期不重叠，公开盖公开）	−1,024	39,996
M3	$\hat{\mathbf{s}}_{1,2}$ 的份 1 换成 32 B 种子 (SHAKE128 展开)	−8,168	**31,828**

65：66,244 → 40,100（-39.5%）；87：84,948 → 49,396（-41.9%）。
n0→n3 只多 +0.21% 指令

思路三类（面试就讲这三句）

1. **能重算就不存**：公开的 $\hat{A}$ 现算；$\mathbf{y}$ 重采（拿暴露换内存）
2. **按生命周期复用**：死缓冲删掉，生命周期不重叠的缓冲共用
3. **份额压成种子**：一份份额 = PRF(seed)，密钥对象 −8 KB

M3 的细节：每系数取 48 位定长 Montgomery 约简（拒绝采样会让 trace 长度随密钥变 ⇒ 弃用）；偏差 $2^{-30.45}$；refresh 先减新份再加旧份（反过来份 0 会短暂等于明文）；新假设 SHAKE128 是 PRF

没做 / 前人：份额打包到 23 位（再省 ≈2.3 KB）、按行重算 $\mathbf{w}$（省 8 KB 但每行重采 $\mathbf{y}$）；Bos–Renes–Sprenkels 的压缩多项式 / 换 NTT 口径不同不比；pqm4 m4fstack 不掩码 5,080 B

masking 的下一步：换场景 【待确认】笔记里查到的是 FIDO2；要不要讲你定

- 新场景（仓库笔记，论文还没写）：**FIDO2 / WebAuthn 安全密钥**，GetAssertion 用 ML-DSA-44 签名；平台 nRF52840（64 MHz Cortex-M4F，256 KB RAM，OpenSK 用的芯片）
- 卖点：FIDO Server Requirements v2.3 草案把 ML-DSA-44/65/87 列为 Recommended；RFC 9964 注册了 COSE 算法号 $-48/-49/-50$
- 真板（只测时间和内存，不测泄漏）：n3 签名 RAM 31,944 B（仿真 31,828，$+0.4\%$）；起初平均 53.7 s
- 延迟优化：掩码 RNG 换 ChaCha20 $\rightarrow$ 24.1 s；再 bitslice A2B / 范数检查 / Decompose / $\mathbf{y}$ 的 B2A $\rightarrow$ **均值 6.23 s，p99 22.6 s**；目标 $\leq$5 s / p99 $\leq$20 s（Firefox 默认 30 s 超时）
- 本质不变：**masking + 省内存**，只是把「为什么要 32 KB 以内」落到一个真实产品约束上
- 和蚂蚁的连接：笔记里备选过「掩码 ML-KEM 解封装做认证扩展」——正好接他们想做的 ML-KEM

Part 4 · HERO

一面已讲，备用

HERO（一面讲过，备用）

- 问题：加密只藏内容，不藏「访问了哪条记录」⇒ ORAM
- 设计：**非对称分布式 ORAM** ——一台主服务器存加密数据库 + 一个**不共谋、资源很少的不可信 helper** 帮忙检索和 eviction；不用 FHE、不要多台全量服务器、不要 TEE
- 安全：主证明 semi-honest，可扩展到 malicious-with-abort
- 实现：OpenSSL，ElGamal 3072-bit MODP + AES-256-CTR；helper 跑在 Raspberry Pi 5（3.15 Gbps 链路）
- 数字：客户端带宽比 Path ORAM 最多降 **25.7**×；每次访问 **0.15 s** vs Onion Ring ORAM 11.91 s；helper 峰值内存 ≈2 MB、代码 ≈105 行；新 eviction 峰值内存比前人小 2–4.9×；发现 triplet eviction 参数设置有问题
- 简历条目里的工程细节：早期 trusted-helper 变体移植 SGX（NTL 链接进 enclave → 改 OpenSSL 路径）；ElGamal 模块化给 FPGA SmartNIC 合作者；Win/Linux/Pi 部署脚本

H/intro.tex:300–334；状态：NDSS 2027 在审

Part 5 · threshold ML-KEM

组里想探索的方向

strict ML-KEM：如果让我做

【待确认】场景是我猜的：threshold 还是 masked? 面试时先问一句「组里说的 ML-KEM 是多方持钥，还是抗侧信道实现?」

FIPS 203 Decaps 骨架（难点都在这）

1. $\mathbf{w} = v - \mathrm{NTT}^{-1}(\hat{\mathbf{s}}^\top \circ \mathrm{NTT}(\mathbf{u}))$ 线性 ⇒ 本地
2. $m' = \mathrm{Compress}_1(\mathbf{w})$ 舍入 = 比较 ⇒ A2B
3. $(K', r') = G(m' \| h)$ SHA3-512 on secret
4. $c' = \mathrm{Encrypt}(ek, m', r')$：CBD 采样（SHAKE → B2A），$A\mathbf{r}$ 线性，$\mathrm{Compress}_{d_u,d_v}$ 又是舍入
5. $\bar{K} = J(z\|c)$；$c \stackrel{?}{=} c'$，不能泄露走了哪条分支 (implicit rejection)

$q = 3329$（12 bit），比 ML-DSA 小，域更小 ⇒ MAC soundness 问题更突出

threshold ML-KEM：SHIELD 打法直接迁移——SPDZ 做 1、4 的线性部分；WRK17 做 Keccak、Compress、比较；daBits 桥接；结果用 oblivious MUX 选 $K'/\bar{K}$

masked ML-KEM：同样的积木——masked Compress (A2B)、masked Keccak、masked CBD（B2A）、**masked comparison**（历史上最容易出错的一块）、FO 的 plaintext-checking-oracle 侧信道

凭记忆的前作：Bos et al. "Masking Kyber" TCHES'21；Heinz et al. first-order masked Kyber on M4 (ePrint 2022/058)；Bhasin et al. masked comparison 攻防 TCHES'21 —— 【待确认】引用前核对

我能带过去的经验：bit-exact 合规的工程纪律；$\mathrm{A2B}_q$（Compress 就是 HighBits 的翻版，q 只有 12 bit 更便宜）、B2A（CBD 采样）；4 份 TI Keccak 直接复用；「只合并 1 个判定位」用在 FO 比较；R1/R2 barrier 与编译器无关可直接迁移

威胁模型差别：KEM 面对**选择密文**，FO 判定位和比较 gadget 是主攻击面；ML-DSA 是已知消息

Part 6 · 概念讲解

我没讲清 / 没意识到的，逐个弄明白

E1 ·ISW 是什么（Ishai–Sahai–Wagner, “Private Circuits: Securing Hardware against Probing Attacks”, CRYPTO 2003）

要解决的问题：攻击者能在芯片上「探」到 t 根导线的值（t-**probing model**，侧信道的理论抽象）。怎么让他探 t 根线也学不到秘密？

做法：把每个比特拆成 $d+1$ 份，$x = x_0 \oplus x_1 \oplus \cdots \oplus x_d$；任意 d 份都是均匀随机 $\Rightarrow$ 探 d 根线什么都看不到

- **XOR / 线性运算**：逐份算，$c_i = a_i \oplus b_i$，免费
- **AND 难**：$a \wedge b = \bigoplus_{i,j} a_i b_j$ 有交叉项 $a_i b_j$，单独算出来就把两份放一起了 $\Rightarrow$ 需要随机数把交叉项遮住 = **ISW AND gadget**

术语：gadget = 一个带掩码的小电路模块；refresh = 用新随机数重新拆份

ISW AND（$d+1$ 份）

1. 对每对 $i < j$ 采随机比特 r_{ij}
2. $r_{ji} = (r_{ij} \oplus a_i b_j) \oplus a_j b_i$　　（括号顺序不能换）
3. $c_i = a_i b_i \oplus \bigoplus_{j \neq i} r_{ij}$

随机数 $d(d+1)/2$ 个；乘法 $(d+1)^2$ 次

2 份例子：$c_0 = a_0 b_0 \oplus r$，
$c_1 = a_1 b_1 \oplus \big((r \oplus a_0 b_1) \oplus a_1 b_0\big)$

和 MPC 的联系（面试金句）：「这就是 GMW 协议里的 AND：每个份额看成一个参与方，交叉项 $a_i b_j$ 由 i、j 两方用共享随机数 r_{ij} 遮住后再合并。」

原论文用 $2t+1$ 份证明；Rivain–Prouff (CHES 2010) 证明 $d+1$ 份的 ISW AND 是 d-probing 安全（并修了 refresh）【待确认】出处凭记忆

E2 ·masking、MPC、MPC-in-the-head 三者关系

	MPC（如 SPDZ、WRK17）	masking（如 ISW、TI）	MPC-in-the-head（IKOS07）
目的	多方联合计算，谁都不看到别人输入	一块芯片抗侧信道	构造 zero-knowledge proof
「参与方」	真实的不同机器	同一芯片上的 $d+1$ 个份额	prover 脑中模拟的 n 个虚拟方
敌手	腐化至多 t 方（半诚实 / 恶意）	探测至多 d 个中间值（probing / noisy leakage）	verifier 要求打开 t 个虚拟方的 view 来检查
能「打开」值吗	能：打开被随机数遮住的值（Beaver 的 d, e；A2B 打开 $x - r$）	基本不能：芯片上任何组合值都可能被探到	打开 t 个 view
完整性	恶意安全靠 IT-MAC	只管隐私，抗故障注入要另做	靠 commit + 随机打开
例子	SHIELD、FLOCK	我的 masked ML-DSA	Picnic、Ligero / Longfellow、ZKBoo

所以「masking 是 local MPC 吗?」——思想上是（ISW AND ≈ GMW AND），但模型更弱的地方正是我论文的发现：

- MPC 的各方物理隔离；芯片上的「各方」共用寄存器、总线、操作数端口，前后两个值的 Hamming distance 会漏（transition），硬件里还有 glitch
- 所以「每一步只碰一份」的 ISW 证明在**值模型**成立，到了编译后的二进制上会被总线打破 ⇒ R1/R2、bus-and-port barrier

IKOS07 = Ishai, Kushilevitz, Ostrovsky, Sahai, "Zero-Knowledge from Secure Multiparty Computation", STOC 2007

E3 · masking 圈的术语（一面那位提到 ISW，老板可能接着问）

- ***d*-th order masking**：$d+1$ 份，抗 d 个探针
- **value model vs transition model**：只看单个值 vs 也看「前一个值 $\oplus$ 后一个值」（寄存器 / 总线被覆写时的 HD 泄漏）。Balasch et al.：n 份在 transition 下只剩 $\lfloor (n-1)/2 \rfloor$ 阶 $\Rightarrow$ 我们检查路径用 3 份
- **glitch**：硬件组合逻辑里信号短暂的毛刺，会让一根线「看到」多份 $\Rightarrow$ 硬件 masking 要 TI / DOM
- **NI / SNI**（Non-Interference / **Strong** Non-Interference, Barthe et al. EUROCRYPT 2016）：gadget 的探针能用多少个输入份模拟。SNI = 只与**内部**探针数有关，与输出探针无关 $\Rightarrow$ 多个 gadget 可以安全**组合**
- **PINI**（Probe-Isolating Non-Interference, Cassiers–Standaert TCHES 2020）：更容易组合的变体
- **TI**（Threshold Implementation, Nikova–Rechberger–Rijmen 2006）：每个输出份只用到部分输入份（**non-completeness**）$\Rightarrow$ 即使有 glitch 也不会一次碰到全部份额。「threshold」不是门限密码
- **DOM**（Domain-Oriented Masking, Groß–Mangard–Korak 2016）：每个份额一个「域」，跨域乘积加随机数后再寄存
- **changing of the guards**（Daemen, CHES 2017）：Keccak χ 的 TI 不用每轮新随机数的技巧
- **refresh**：用新随机数重拆份；我们用的 RP10 式线性 refresh 一阶均匀，但**不是 SNI**
- **TVLA**（Test Vector Leakage Assessment）：固定输入 vs 随机输入两组 trace 做 Welch t-test，$|t|$ 超阈值 = 检出泄漏
- **CPA**（Correlation Power Analysis）/ **template attack**：真正去恢复密钥的攻击

E4 ·SPDZ 怎么做到 malicious，以及 MPC 里的 A2B

IT-MAC（information-theoretic MAC）

- 全局 MAC key α 被加法拆给各方，谁也不知道
- 每个秘密 x：各方持 (x_i, m_i)，$\sum x_i = x$，$\sum m_i = \alpha x$
- 恶意方把 x 改成 $x+\delta$，还得把 MAC 改成 $\alpha(x+\delta)$ ——不知道 α，只能猜中概率 $1/p \Rightarrow$ **MAC check** 时被抓 $\Rightarrow$ abort
- 所以 p 小（$q \approx 2^{23}$）时要多个 α 或扩域

Beaver triple：预处理好 $([a], [b], [c = ab])$；算 $[xy]$ 时打开 $d = x - a, e = y - b$（均匀随机，不泄露），$[xy] = [c] + d[b] + e[a] + de \Rightarrow$ online 一轮

MPC 里的 A2B（对照 masking 的 A2B）

- 预处理一个 **daBit / edaBits**：同一个随机数 r 同时有算术份 $[r]_A$ 和逐比特布尔份 $[r_0], \ldots, [r_{k-1}]_B$
- 打开 $c = x - r$（被 r 遮住，公开无害）
- 布尔电路里算 $x = c + r$（加法器 + mod p 修正）
- edaBits = Escudero et al. CRYPTO 2020，「一次拿一串比特」比逐比特 daBit 省

对照：MPC 能「打开 $x - r$」，所以 A2B 只要一个公开数 + 一个加法器；芯片上不能打开，只能用 SecAdd 在份额上整体算 $\Rightarrow$ masking 的 A2B 更贵

abort 的种类：security-with-abort（我们）< identifiable abort（能指出谁作弊）< robustness / guaranteed output delivery

E5 ·为什么拒绝采样让 threshold 化这么难（malicious 的根本理由）

Fiat–Shamir with aborts（ML-DSA 的核心）

- $\mathbf{z} = \mathbf{y} + c\mathbf{s}_1$；$\mathbf{y}$ 是随机 mask
- 若 $\|\mathbf{z}\|_\infty \geq \gamma_1 - \beta$ 就**拒绝重来**
- 这样**被接受**的 $\mathbf{z}$ 分布与 $\mathbf{s}_1$ 无关；**被拒绝**的 $\mathbf{z}$ 是「往 $c\mathbf{s}_1$ 方向偏」的样本 $\Rightarrow$ 泄露 $\mathbf{s}_1$
- 期望重试：$44/65/87 = 4.25$ / 5.1 / 3.85 次

threshold 的难点：多方要在**不暴露候选** $\mathbf{z}$ 的前提下共同决定拒不拒绝。各路线的回答：

- SHIELD：整个检查在 MPC 里，只公开 1 个 reject bit
- Celi：短份额 + 各方本地拒绝（改分布）
- Quorus：加噪声 e_w，使被拒的 w_1 可安全公开

为什么半诚实不够（3 个具体攻击）

1. **篡改判定**：恶意方在 $\mathbf{z}$ 的份或 reject bit 上加 δ $\Rightarrow$ 本该拒绝的 $\mathbf{z}$ 被打开 $\Rightarrow$ 泄露 $\mathbf{s}_1$
2. **selective failure**：让「是否 abort」依赖某个秘密比特；因为拒绝本来就常见，人为 abort 和正常拒绝分不出 $\Rightarrow$ 多次会话逐比特提取。$\Rightarrow$ 影响后续行为的值必须**先认证再用**
3. **nonce 偏置 / rushing**：看到别人的贡献再选自己的 $\Rightarrow$ $\mathbf{y}$ 有偏（Celi 用「先 commit 哈希再公开」防）

一句话：「threshold 签名里持钥方本身就是敌手，而 ML-DSA 的安全恰恰靠『被拒的东西永远不出现』——半诚实模型保证不了这一点。」

E6 ·几种 secret sharing 一页看懂

	怎么分	优点	代价 / 谁用
additive（n-of-n）	$x = \sum x_i$	最简单；配 IT-MAC 就是 SPDZ，容忍 $n-1$ 恶意	少一方就签不了；SHIELD
Shamir（t-of-n）	$x = f(0)$，f 是 $t-1$ 次多项式，各方拿 $f(i)$	任意 t 方能重构；可扩展到很多方	份额在 $\mathbb{Z}_q$ 均匀、Lagrange 系数大 $\Rightarrow$ 不能本地做拒绝；乘法要 honest majority；Quorus
replicated (RSS)	拆成多块加法份，每块给一组人	重构系数 0/1；乘法本地化	份数 $\binom{N}{T-1}$ 随 N 指数涨
short RSS	RSS，但每块都是短的 ML-DSA 秘密	每方的部分秘密本身就短 $\Rightarrow$ 能本地拒绝采样	$N \leq 6$；Celi / Mithril

一句话串起来：「additive + MAC 给最强的威胁模型但只能 n-of-n；Shamir 给 t-of-n 但要 honest majority；short RSS 让拒绝采样可以本地做，但份数爆炸。我们选第一种是因为 FPKI 要 n-of-n 且容忍 $n-1$。」

术语别说错 + 数字速查

术语

- masking $\neq$ MPC-in-the-head。MPC-in-the-head 是 IKOS07 用「脑中模拟 MPC」构造 ZK 证明（Picnic、Longfellow 那类）。masking 应该说：**「单设备上的 MPC / ISW private circuits，probing model 下每个中间值拆成 $d+1$ 份」**
- A2B / B2A：arithmetic $\leftrightarrow$ Boolean masking 转换
- short RSS（Celi）vs Shamir（Quorus）vs additive + IT-MAC（SPDZ）
- security-with-abort / identifiable abort / robustness
- functional interchangeability vs algorithmic compliance

数字

- ML-DSA $q = 8380417 = 2^{23} - 2^{13} + 1$；ML-KEM $q = 3329$
- 期望重试：44/65/87 = 4.25 / 5.1 / 3.85
- SHIELD：2–5 方；单次迭代 198–536 ms；23 轮；7.7–14.7 MB/CA
- WRK17 online 4 轮；SampleInBall 890K$\rightarrow$355K（相对最初编码）
- Celi：$\leq$6 方，3 轮/次，LAN $<$ 20 ms
- Quorus：honest maj.，测到 63 方
- masking：峰值 RAM $-37.1\%/-39.5\%/-41.9\%$；44: 50,612 $\rightarrow$ 31,828 B
- HERO：带宽 25.7$\times$，0.15 s/访问
- FLOCK：9 轮，4 方 25 ms